

Module 1 — Raw Sequencing Reads and Quality Control

Status: built.

Curriculum module: Module 1 — Raw Sequencing Reads and Quality Control.

Guiding question

Given a set of raw sequencing reads, how do I confirm they are fit for downstream analysis before I invest compute time in them?

Learning outcomes

- Explain FASTQ record structure and Phred-quality (Q-score) encoding.
- Retrieve a selected public run and verify file/cache integrity before extracting it.
- Interpret per-base quality, adapter content, duplication, and length/composition summaries diagnostically — not just run the tool and glance at PASS/FAIL colors.
- Distinguish diagnostic QC (FastQC — reports, never modifies data) from automatic trimming (fastp — acts on the evidence).
- Process paired-end reads without breaking read pairing, and verify that pairing directly rather than assuming a tool preserved it.
- Document preprocessing decisions, compare before/after metrics, and preserve a full command/audit log.

Portfolio artifact (per curriculum)

A QC report that documents: data source and checksums, before/after QC figures, the trimming rationale, the full command log, and explicit acceptance criteria for "reads are ready to align."

Scripts in this module

Each script adds exactly one new idea on top of the previous one, all built around the same real paired-end study used elsewhere in this repository for continuity: **PRJNA1518998** ("Effect of adipocyte depletion on the retina," *Mus musculus*, Vanderbilt University Medical Center; GEO accession GSE345194; real runs SRR40359383 and SRR40359384).

#	File	New idea introduced
1	<u>scripts/1_Fetch_And_Verify_Raw_Reads.sh</u>	Retrieve one hardcoded run with <code>prefetch/fasterq-dump</code> , validate the SRA cache with <code>vdb-validate</code> <i>before</i> extracting, and checksum the resulting FASTQ mates. Includes a FASTQ-record and Phred+33 explainer.

#	File	New idea introduced
2	<u>scripts/2_Interpret_FastQC_Diagnostics.sh</u>	Run FastQC on that sample, then <i>parse</i> its <code>summary.txt</code> PASS/WARN/FAIL verdicts programmatically and interpret the three most commonly misread modules (per-base quality, adapter content, duplication).
3	<u>scripts/3_Batch_MultiQC_Summary.sh</u>	Loop the fetch+FastQC steps fault-tolerantly over both real runs in the study, and aggregate them into one comparison report with MultiQC.
4	<u>scripts/4_Evidence_Based_Fastp_Trimming.sh</u>	Read the FastQC evidence and <i>decide</i> which fastp flags are justified (not apply them by default), run fastp, write a plain-language <code>THRESHOLD_RATIONALE.md</code> , and re-run FastQC on the trimmed output.
5	<u>scripts/5_Paired_End_Integrity_Check.sh</u>	Verify paired-end integrity two ways: matching mate read counts , and matching mate read ID order — the second check catches desynchronized mates that the first one misses. Checks both raw and trimmed reads.
6	<u>scripts/6_Automated_QC_Pipeline_AllSamples.sh</u>	Integrate scripts 1–5 into one fault-tolerant <code>sbatch</code> pipeline across every sample — the reusable, standalone <code>qc_pipeline.sh</code> this module previously lacked.
7	<u>scripts/7_QC_Report_Generator.sh</u>	Auto-assemble every artifact scripts 1–6 produced (checksums, before/after QC, rationale, pairing verdicts, command logs) into the single <code>QC_REPORT.md</code> portfolio artifact the curriculum requires, with a hand-completed acceptance-criteria table and reflection questions.

All seven scripts were validated with `bash -n`. Scripts 5 and 7's core logic (pairing-integrity detection and report assembly) were also functionally smoke-tested against synthetic FASTQ/report fixtures — matched pairs, count mismatches, and read-ID-order mismatches were all correctly identified as PASS/FAIL.

How to run

See [RUNBOOK.md](#) for the full step-by-step execution order, expected outputs, and troubleshooting. In short: run scripts 1–5 once on the login node/via `sbatch` to learn each stage individually, then use script 6 as the actual reusable pipeline for a full run, and finish with script 7 (login node, no Slurm needed) to generate the portfolio report.

What this replaces

The previous version of this module was a **planned stub** noting that QC mechanics were already exercised inline inside modules 2/4/6/7's pipeline scripts, but not yet packaged as its own standalone, reusable teaching module. Those inline QC steps in other modules are untouched by this work — this module adds the missing dedicated lesson plan, diagnostic-interpretation teaching, evidence-based trimming decision process, and portfolio-artifact generator that a standalone Module 1 requires.

Teaching materials

- Lecture deck: [lecture/Raw_Reads_QC_Lecture.pptx](#).
- Runbook: [RUNBOOK.md](#).
- Still to develop per [Section 5 of the curriculum](#): a guided student workbook and a transfer-task assessment using a different accession/organism.